Documentation report

1. **Problem Framing and Dataset Analysis**

   Our dataset consists of the 2019 NYC Yellow Taxi trip records, the taxi zone lookup file, and the taxi zone shapefile. The goal of our dashboard is to analyze urban mobility patterns to inform city planning, with a focus on congestion and speed.

   The Data Challenges:

   - **Scale:** The file size of the NYC Yellow Taxi trip records file was approximately 680MB, which exceeds the standard memory limit for applications like Excel, so we decided to use Pandas with the chunking approach to inspect the file.
   - **Data Issues:** When inspecting the large dataset, we discovered that some records were impossible (negative trip duration, zero distance, and invalid zones).
   - **Missing fields:** We noted that the congestion surcharge field was largely empty or had a value of zero. We concluded that it was not a useful field for our analysis.

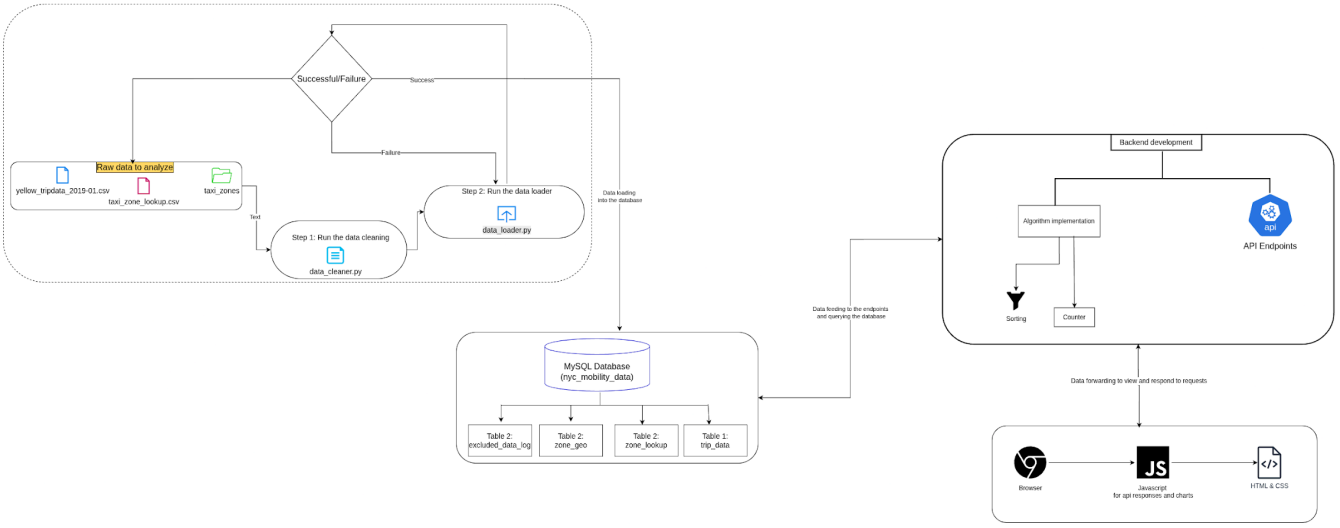
   Assumptions:

   - **Passenger count:** We assumed that the maximum passenger count should be 6 and the minimum passenger count is 1 since a trip cannot exist without a passenger.
   - **Trip Duration:** We assumed that a trip cannot last more than four hours so that we can leave out trips that are too long.

System architecture and design decisions

System Architecture

[Link to system architecture]



Schema Design

- Our database implements a Star Schema architecture to handle the 680MB trip dataset efficiently. By separating high-frequency trip data from static metadata (zone_lookup), we reduced redundancy and optimized storage.
    - trip_data: This is the central hub of our analysis. It stores millions of records but uses data types like TINYINT and DECIMAL to help reduce disk space.
    - zone_lookup & zone_geo: We separated the zone names from their spatial geometry to ensure that search by name queries in the zone_lookup table remain fast and not slowed down by spatial strings stored as LONGTEXT in the zone_geo table.
    - excluded_data_log: This table captures the log of rows that failed our validation rules which helped us with transparent data quality reporting.
- When designing the schema, we implemented some critical decisions

- Feature engineering programmatically: When we discovered that the raw congestion_surcharge column was largely empty, we decided to go with a more analytical approach and programmatically derived congestion_level, avg_speed_mph, is_peak_hour, and fare_per_mile during the process of loading the cleaned data into the database.
- Extra data integrity: We implemented database constraints (CHECK and FOREIGN KEY) to act as a secondary defense in case the cleaning script allows a logical error, the database will reject impossible data, like a trip ending before it started.
- Performance Optimization: To ensure real-time performance, we implemented specialized indexing, like the idx_od_pair on both pickup and dropoff locations, to help pickup and destination queries run faster.

1. Algorithm logic and data structures

In this project, we implemented two algorithms that were built without using any built-in Libraries where the data is counted and sorted manually.

➢ Algorithm 1 : Manual counting

This counts the trips per zone where a dictionary is used to keep counts as we iterate through all trips records exactly once.

❖ Pseudocode (the code in plain english):

Function count_zones_manual(zone_ids):

- Create empty dictionary zone_counts

- For each zone_id in the dataset
    - If we already have the zone_id, add 1 to its count
    - If we not see the zone_id, add it to the dictionary and set its count to 1
    - Then return zone_counts dictionary

Example

input : [237, 161,237]

With this example of zone ids the algorithm iterates through each once

Iteration1 : zone_id = 237 as it didn't exist it set one to its count so that it Becomes a dictionary with zone_counts = {237:1}

Iteration 2: zone_id = 161 as it is also not in the dictionary we set 1 to its count so that it becomes zone_counts = [237:1 , 161:1]

Iteration 3 : zone_id =237 as this zone_id already exists in the dictionary we add one to its count so that it becomes zone_counts = [237:2 , 161:1]

❖ **Time complexity analysis:**

o(n): we iterate through all n trip records once

❖ **Space complexity analysis :**

o(m): dictionary stores one entry per unique zone

This algorithm is efficient because it is most effective for counting and Simple to implement.

➢ **Algorithm 2: Selection sort**

This algorithm sorts zones by trip count from the highest to lowest. It repeatedly finds the maximum value in the unsorted and puts it in the front.

❖ **Pseudocode:**

- Look at every every position i
- Assume that i is the max (the highest)

- Look at numbers after i
- If you find a bigger number than current max, remember its position
- Swap the current position i with max index
- Return sorted list

Example:
Input : [(161, 3892), (237, 5555)]
In this example , the first step is to find the maximum in the positions which is at index 1 with the zone_id 237 and then indexes are swapped
Result: [(237, 5555), (161,3892)]

❖ **Time complexity analysis**

- $O(n^2)$ *which sorts in place and uses few variables*

This algorithm is efficient because it is simple to implement and $O(n^2)$ is acceptable for small numbers.

2. Insights and interpretation

We demonstrated that peak hours are not the efficient ones because they are the busy hours of the day in New York be it morning or evening rush. As these peak hours reflect increase in trip volumes but are inefficient in performance based on efficiency metrics we derived or got from the data set. Below are the insights we got to form from the data set;

1. Peak hours vs Efficiency

Trip volumes spike up during peak hours ranging from 8am to 10am and from 6pm to 8pm, but the demand doesn't match efficiency as then there are low average speeds and reduced fare per miles due to congestion levels that are high which increases travel time. Peak hours look profitable to drivers due to volume of trips spiking up but are inefficient when measured.

So if all hours could share the trip volumes as in be redistributed it can improve the efficiency by increasing fare per mile or reduce congestion levels.

2. **Short trip and their congestion effects**

Short trips during peak hours due to congestion levels that are high generate lowest fare per mile amounts while increasing travel time. So we discovered that short trips are slower, earn less and occupy the road infrastructure. For short trips people can be encouraged to walk, use cycling or implement public transport.

3. **Borough efficiency**

Some boroughs dominate in total trips and revenue, but it performs poorly on efficiency metrics. This suggests that congestion concentration, not demand itself drives inefficiency. Improving different borough connectivity and incentivising travel redistribution could balance load across the network and improve system-wide performance.

4. **Tipping as a quality signal**

Higher tips correlate with off-peak travel, lower congestion, longer trip distances, and better speeds. Passengers appear to reward smooth and efficient journeys. This indicates that congestion harms not only operational efficiency but also passenger satisfaction and driver income.

5. **Passenger load hotspots**

Zones with consistently high passenger counts also experience high peak-hour volumes. These areas demonstrate existing demand for shared capacity rather than adding more individual taxi trips, city planners could prioritise shared ride programs or bus route reinforcement in these hotspots.

Overall Interpretation

One conclusion is consistently supported by the evidence: rush hour congestion in New York City lowers efficiency at every level of the transportation ecosystem, regardless of time, geography, passenger behavior, and load distribution. Ineffective demand distribution is the problem, not just high demand. The city could simultaneously increase driver earnings, improve passenger satisfaction, and reduce congestion by distributing peak traffic, discouraging ultra-short car trips, enhancing outer borough connectivity, and investing in

shared transit in high-load zones. The information presents a convincing case for more intelligent urban transportation planning rather than merely describing taxi activity.